

Monitoring and Observability

Even though I put observability in its own section, observability should be integral to the design of a product, rather than an afterthought. The more complex a product, the more crucial observability is.

Observability is a universal practice across all software engineering disciplines. It's a big industry with established best practices and many ready-to-use proprietary and open source solutions.⁴ To avoid reinventing the wheel, I'll focus on what's unique to applications built on top of foundation models. The book's [GitHub repository](#) contains resources for those who want to learn more about observability.⁵

4 As of this writing, the aggregated market capitalization of a few of the largest observability companies (Data-dog, Splunk, Dynatrace, New Relic) is close to \$100 billion.

5 My book, *Designing Machine Learning Systems* (O'Reilly, 2022), also has a chapter on monitoring. An early draft of the chapter is available on my blog at "[Data Distribution Shifts and Monitoring](#)".

The goal of monitoring is the same as the goal of evaluation: to mitigate risks and discover opportunities. Risks that monitoring should help you mitigate include application failures, security attacks, and drifts. Monitoring can help discover opportunities for application improvement and cost savings. Monitoring can also help keep you accountable by giving visibility into your system's performance.

Three metrics can help evaluate the quality of your system's observability, derived from the DevOps community:

- **MTTD (mean time to detection):** When something bad happens, how long does it take to detect it?
- **MTTR (mean time to response):** After detection, how long does it take to be resolved?
- **CFR (change failure rate):** The percentage of changes or deployments that result in failures requiring fixes or rollbacks. If you don't know your CFR, it's time to redesign your platform to make it more observable.

Having a high CFR doesn't necessarily indicate a bad monitoring system. However, you should rethink your evaluation pipeline so that bad changes are caught before being deployed. Evaluation and monitoring need to work closely together. Evaluation metrics should translate well to monitoring metrics, meaning that a model that does well during evaluation should also do well during monitoring. Issues detected during monitoring should be fed to the evaluation pipeline.

Monitoring Versus Observability

Since the mid-2010s, the industry has embraced the term “observability” instead of “monitoring.” Monitoring makes no assumption about the relationship between the internal state of a system and its outputs. You monitor the external outputs of the system to figure out when something goes wrong inside the system—there's no guarantee that the external outputs will help you figure out what goes wrong.

Observability, on the other hand, makes an assumption stronger than traditional monitoring: that a system's internal states can be inferred from knowledge of its external outputs. When something goes wrong with an observable system, we should be able to figure out what went wrong by looking at the system's logs and metrics without having to ship new code to the system. Observability is about instrumenting your system in a way that ensures that sufficient information about a system's run-time is collected and analyzed so that when something goes wrong, it can help you figure out what goes wrong.

In this book, I'll use the term “monitoring” to refer to the act of tracking a system's information and “observability” to refer to the whole process of instrumentating, tracking, and debugging the system.

Metrics

When discussing monitoring, most people think of metrics. However, metrics themselves aren't the goal. Frankly, most companies don't care what your application's output relevancy score is unless it serves a purpose. The purpose of a metric is to tell you when something is wrong and to identify opportunities for improvement.

Before listing what metrics to track, it's important to understand what failure modes you want to catch and design your metrics around these failures. For example, if you don't want your application to hallucinate, design metrics that help you detect hallucinations. One relevant metric might be whether an application's output can be inferred from the context. If you don't want your application to burn through your API credit, track metrics related to API costs, such as the number of input and output tokens per request or your cache's cost and your cache's hit rate.

Because foundation models can generate open-ended outputs, there are many ways things can go wrong. Metrics design requires analytical thinking, statistical knowledge, and, often, creativity. Which metrics you should track are highly application-specific.

This book has covered many different types of model quality metrics (Chapters 4–6, and later in this chapter) and many different ways to compute them (Chapters 3 and 5). Here, I'll do a quick recap.

The easiest types of failures to track are format failures because they are easy to notice and verify. For example, if you expect JSON outputs, track how often the model outputs invalid JSON and, among these invalid JSON outputs, how many can be easily fixed (missing a closing bracket is easy to fix, but missing expected keys is harder).

For open-ended generations, consider monitoring factual consistency and relevant generation quality metrics such as conciseness, creativity, or positivity. Many of these metrics can be computed using AI judges.

If safety is an issue, you can track toxicity-related metrics and detect private and sensitive information in both inputs and outputs. Track how often your guardrails get triggered and how often your system refuses to answer. Detect abnormal queries to your system, too, since they might reveal interesting edge cases or prompt attacks.

Model quality can also be inferred through user natural language feedback and conversational signals. For example, some easy metrics you can track include the following:

- How often do users stop a generation halfway?
- What's the average number of turns per conversation?
- What's the average number of tokens per input? Are users using your application for more complex tasks, or are they learning to be more concise with their prompts?
- What's the average number of tokens per output? Are some models more verbose than others? Are certain types of queries more likely to result in lengthy answers?
- What's the model's output token distribution? How has it changed over time? Is the model getting more or less diverse?

Length-related metrics are also important for tracking latency and costs, as longer contexts and responses typically increase latency and incur higher costs.

Each component in an application pipeline has its own metrics. For example, in a RAG application, the retrieval quality is often evaluated using context relevance and context precision. A vector database can be evaluated by how much storage it needs to index the data and how long it takes to query the data.

Given that you'll likely have multiple metrics, it's useful to measure how these metrics correlate to each other and, especially, to your business north star metrics, which can be DAU (daily active user), session duration (the length of time a user spends actively engaged with the application), or subscriptions. Metrics that are strongly correlated to your north star might give you ideas on how to improve your north star. Metrics that are not at all correlated might also give you ideas on what not to optimize for.

Tracking latency is essential for understanding the user experience. Common latency metrics, as discussed in [Chapter 9](#), include:

- Time to first token (TTFT): the time it takes for the first token to be generated.
- Time per output token (TPOT): the time it takes to generate each output token.
- Total latency: the total time required to complete a response.

Track all these metrics per user to see how your system scales with more users.

You'll also want to track costs. Cost-related metrics are the number of queries and the volume of input and output tokens, such as tokens per second (TPS). If you use an API with rate limits, tracking the number of requests per second is important to ensure you stay within your allocated limits and avoid potential service interruptions.

When calculating metrics, you can choose between spot checks and exhaustive checks. Spot checks involve sampling a subset of data to quickly identify issues, while exhaustive checks evaluate every request for a comprehensive performance view. The choice depends on your system's requirements and available resources, with a combination of both providing a balanced monitoring strategy.

When computing metrics, ensure they can be broken down by relevant axes, such as users, releases, prompt/chain versions, prompt/chain types, and time. This granularity helps in understanding performance variations and identifying specific issues.

Logs and traces

Metrics are typically aggregated. They condense information from events that occur in your system over time. They help you understand, at a glance, how your system is doing. However, there are many questions that metrics can't help you answer. For example, after seeing a spike in a specific activity, you might wonder: "Has this happened before?" Logs can help you answer this question.

If metrics are numerical measurements representing attributes and events, logs are an append-only record of events. In production, a debugging process might look like this:

1. Metrics tell you something went wrong five minutes ago, but they don't tell you what happened.
2. You look at the logs of events that took place around five minutes ago to figure out what happened.
3. Correlate the errors in the logs to the metrics to make sure that you've identified the right issue.

For fast detection, metrics need to be computed quickly. For fast response, logs need to be readily available and accessible. If your logs are 15 minutes delayed, you will have to wait for the logs to arrive to track down an issue that happened 5 minutes ago.

Because you don't know exactly what logs you'll need to look at in the future, the general rule for logging is to log everything. Log all the configurations, including the model API endpoint, model name, sampling settings (temperature, top-p, top-k, stopping condition, etc.), and the prompt template.

Log the user query, the final prompt sent to the model, the output, and the intermediate outputs. Log if it calls any tool. Log the tool outputs. Log when a component starts, ends, when something crashes, etc. When recording a piece of log, make sure to give it tags and IDs that can help you know where this log comes from in the system.

Logging everything means that the amount of logs you have can grow very quickly. Many tools for automated log analysis and log anomaly detection are powered by AI.

While it's impossible to process logs manually, it's useful to manually inspect your production data daily to get a sense of how users are using your application. [Shankar et al., \(2024\)](#) found that the developers' perceptions of what constitutes good and bad outputs change as they interact with more data, allowing them to both rewrite their prompts to increase the chance of good responses and update their evaluation pipeline to catch bad responses.

If logs are a series of disjointed events, traces are reconstructed by linking related events together to form a complete timeline of a transaction or process, showing how each step connects from start to finish. In short, a trace is the detailed recording of a request's execution path through various system components and services. In an AI application, tracing reveals the entire process from when a user sends a query to when the final response is returned, including the actions the system takes, the documents retrieved, and the final prompt sent to the model. It should also show how much time each step takes and its associated cost, if measurable. [Figure 10-11](#) is a visualization of a request's trace in [LangSmith](#).

Ideally, you should be able to trace each query's transformation step-by-step through the system. If a query fails, you should be able to pinpoint the exact step where it went wrong: whether it was incorrectly processed, the retrieved context was irrelevant, or the model generated a wrong response.

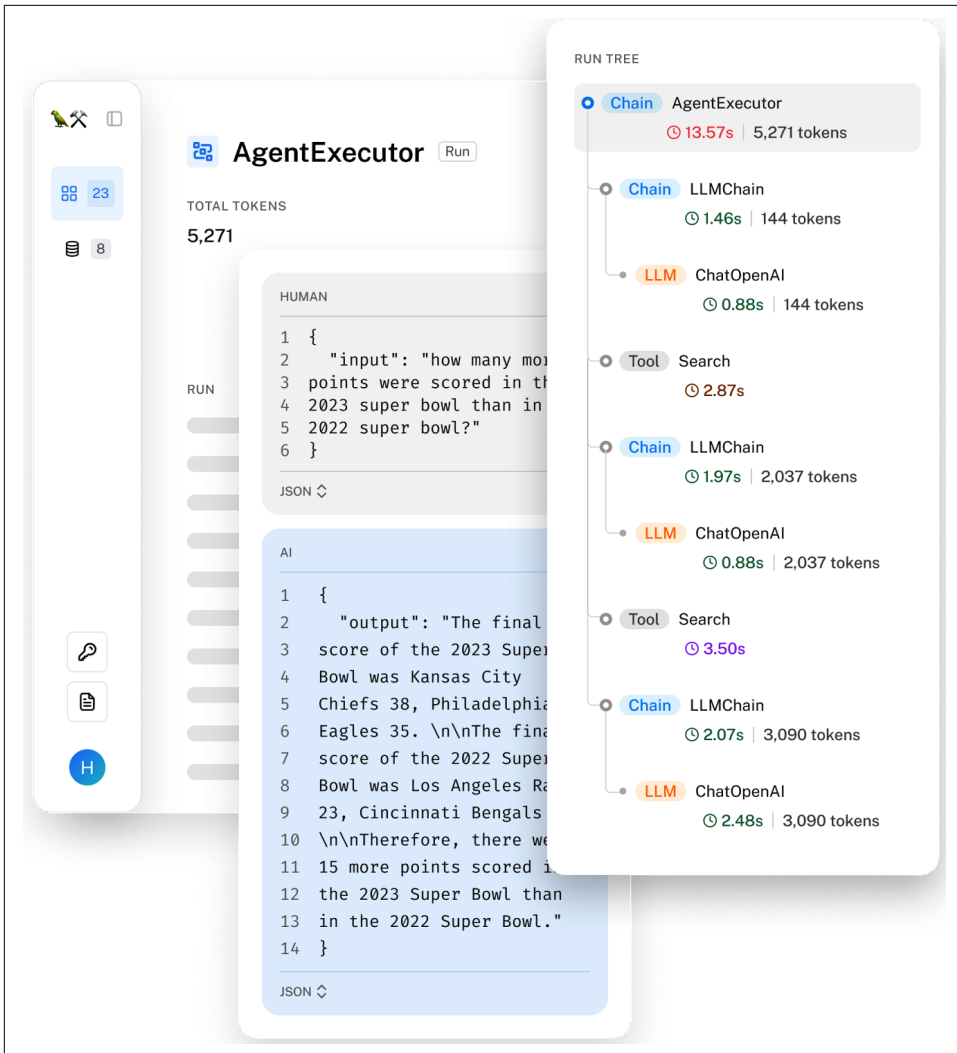


Figure 10-11. A request trace visualized by LangSmith.

Drift detection

The more parts a system has, the more things that can change. In an AI application these can be:

System prompt changes

There are many reasons why your application's system prompt might change without your knowing. The system prompt could've been built on top of a prompt template, and that prompt template was updated. A coworker could've

found a typo and fixed it. A simple logic should be sufficient to catch when your application's system prompt changes.

User behavior changes

Over time, users adapt their behaviors to the technology. For example, people have already figured out how to frame their queries to get better results on Google Search or how to make their articles rank higher on search results. People living in areas with self-driving cars have already figured out how to bully self-driving cars into giving them the right of way (Liu et al., 2020). It's likely that your users will change their behaviors to get better results out of your application. For example, your users might learn to write instructions to make the responses more concise. This might cause a gradual drop in response length over time. If you look only at metrics, it might not be obvious what caused this gradual drop. You need investigations to understand the root cause.

Underlying model changes

When using a model through an API, it's possible that the API remains unchanged while the underlying model is updated. As mentioned in [Chapter 4](#), model providers might not always disclose these updates, leaving it to you to detect any changes. Different versions of the same API can have a significant impact on performance. For instance, [Chen et al. \(2023\)](#) observed notable differences in benchmark scores between the March 2023 and June 2023 versions of GPT-4 and GPT-3.5. Likewise, Voiceflow reported a [10% performance drop](#) when switching from the older GPT-3.5-turbo-0301 to the newer GPT-3.5-turbo-1106.

AI Pipeline Orchestration

An AI application can get fairly complex, consisting of multiple models, retrieving data from many databases, and having access to a wide range of tools. An orchestrator helps you specify how these different components work together to create an end-to-end pipeline. It ensures that data flows seamlessly between components. At a high level, an orchestrator operates in two steps, components definition and chaining:

Components definition

You need to tell the orchestrator what components your system uses, including different models, external data sources for retrieval, and tools that your system can use. A model gateway can make it easier to add a model.⁶ You can also tell the orchestrator if you use any tools for evaluation and monitoring.

⁶ Because of this, some orchestrator tools want to be gateways. In fact, so many tools seem to want to become end-to-end platforms that do everything.

Chaining

Chaining is basically function composition: it combines different functions (components) together. In chaining (pipelining), you tell the orchestrator the steps your system takes from receiving the user query until completing the task. Here's an example of the steps:

1. Process the raw query.
2. Retrieve the relevant data based on the processed query.
3. Combine the original query and the retrieved data to create a prompt in the format expected by the model.
4. The model generates a response based on the prompt.
5. Evaluate the response.
6. If the response is considered good, return it to the user. If not, route the query to a human operator.

The orchestrator is responsible for passing data between components. It should provide toolings that help ensure that the output from the current step is in the format expected by the next step. Ideally, it should notify you when this data flow is disrupted due to errors such as component failures or data mismatch failures.



An AI pipeline orchestrator is different from a general workflow orchestrator, like Airflow or Metaflow.

When designing the pipeline for an application with strict latency requirements, try to do as much in parallel as possible. For example, if you have a routing component (deciding where to send a query) and a PII removal component, both can be done at the same time.

There are many AI orchestration tools, including [LangChain](#), [LlamaIndex](#), [Flowise](#), [Langflow](#), and [Haystack](#). Because retrieval and tool use are common application patterns, many RAG and agent frameworks are also orchestration tools.

While it's tempting to jump straight to an orchestration tool when starting a project, *you might want to start building your application without one first*. Any external tool brings additional complexity. An orchestrator can abstract away critical details of how your system works, making it hard to understand and debug your system.